* Number System & Bitwise Operator *

1] Introduction to Number System :-

1) It is just a system to represent numbers in different form with different base values.
2) We use decimal number system in our day to day life. Computer uses binary number system.
3) There are octal & hexadecimal number system as well. hexadecimal number system used in colour codes & also in memory addresses in computer.

2] Decimal Number System :-

1] In decimal number system 0-9 eligits are used. It has base 10
2] Let's understand base 10 means like we can represent any number in the power of 10.

e.g. $(3451)_{10} = 3\times10^3 + 4\times10^2 + 5\times10^1 + 1\times10^0$
$= 3451$

So,
any decimal numbe can be represented in the powers of 10.

3] Binary Number System :

1] It uses only 0 & 1

2] Base of binary is 2

3] e.g. →

$(1001)_2 = (1 \times 2^3) + (0 \times 2^2) + (0 \times 2^1) + (1 \times 2^0)$

$= 8 + 0 + 0 + 1$

$= 9$ ← This is decimal number

4] Conversion of Binary to Decimal :-

lets convert 1011 into decimal

$(1011)_2 = (1 \times 2^3) * (0 \times 2^2) + (1 \times 2^1) + (1 \times 2^0)$

$= 8 + 0 + 2 + 1$

$= 11$

we will see program for it later in loops

5] Conversion of Decimal to Binary :-

lets take a number 11

2	11	1
2	5	1
2	2	0
	1	

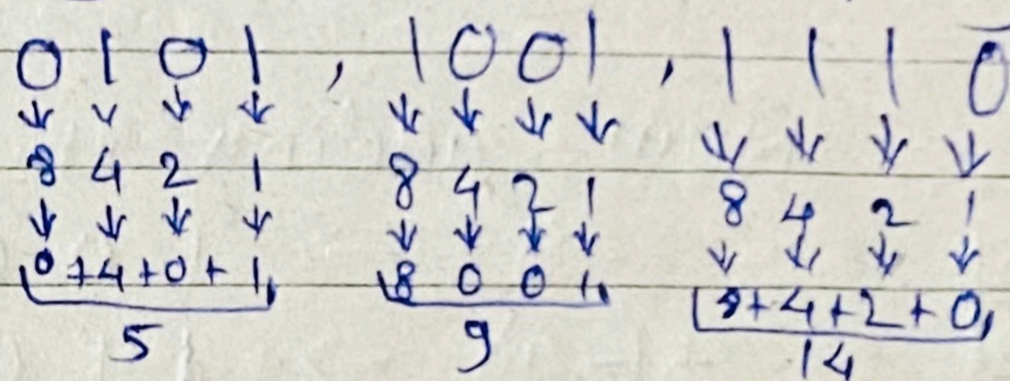
So, Binary of 11 is 1011

5 Binary addition & Binary Substraction

a] Binary addition :-

```
  1
 0101 → 5
+1001 → 9
 1110 → 14
```

Here, we can simply calculate, decimal of any number, from 0101, 1001, 1110



Here, 1+1=2 & binary of 2 is 10 so we take 0 first & took 1 carry
also we can add as many as 0 in front of number

```
  1 1 1
 0 1 1 0 → 14
+0 1 0 1 1 → 11
 1 1 0 0 1 → 25
```

```
  1 1 1 1
  1 1 1 1 →  8+4+2+1 → 15
+ 1 1 1 0 →  8+4+2+0 → 14
 1 1 1 0 1 → 16+8+4+0+1  29.
```

b] Binary Substraction :-

- In binary substraction we have to perform addⁿ of given two number, like,
- 9-4 = 5 same way 9+(-4) = 5 as well
- So we have to take negation of the second number
- 9 → 1001 →
  +(-4) → +1100
  5
- To make second number negative we have to use, 2's complement.
- 2's complement is, steps are,
  i] swap the bits
  ii] add 1 in swapped bits
- We can add as many as zeros here as well

So, negative of 4 can be do like this

binary of 4 is 100

negation/swapping adding 1 in it

```
000001 1 1   Step 1 (Here we can place as 0 as it will be in negative)
+111110 0 1  Step 2 (like this)
111111 0 0   (It will still remain negative)
```

Now, adding this 2's complement in 9

```
  9  → 1 0 0 1 → 9
+(-4) → + 1 1 0 0 → -4
  5     0 1 0 1 → 5
```

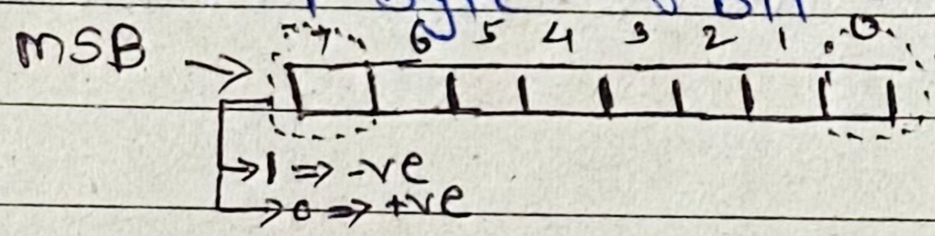
In bigger picture,

```
0 0 0 0 0 0 0 0 1 0 0 1
1 1 1 1 1 1 1 1 1 1 0 0
0 0 0 0 0 0 0 0 0 1 0 1
```

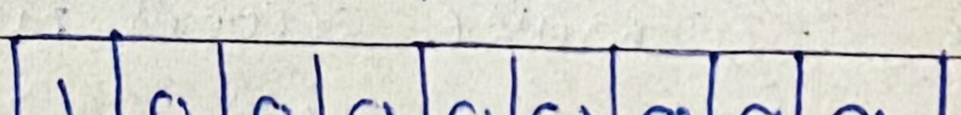
until it get eliminate from memory

7] Why 128 in byte is -128 ?:-

- int a = 128;
  byte b = (byte) a; ⇒ -128
  we explicitly convert int into byte but why it gave -128
- 1 byte = 8 bit



MSB = most significant bit
LSB = least significant bit
in MSB, if 1 is present it shows number is negative & if 0 is present it shows number is positive

- for 128, binary is,
  10000000 i.e. 
  here MSB is 1 i.e. negative in the byte. but in int we have 4 byte or 32 bit memory in that case MSB is 0 & 128 become +ve

128 represents in int here MSB is 0 so number is +ve

- if we try to add 1 in 127 then this will happen

```
01111111 → 127
+00000001 → 1
10000000 → 128
```

MSB here is 1 so it become negative

8] Storing Positive Numbers in memory:

byte num = 42

2	42	0
2	21	1
2	10	0
2	5	1
2	2	0
	1	

So in memory value of num is stored as
101010

while retrieving,
System.out.println(num);
these binaries will get read & then get converted into decimal

9] Storing Negative Numbers in memory:

byte num = -42

take 2's complement of binary of 42 = 00101010

1's = swap number = 11010101

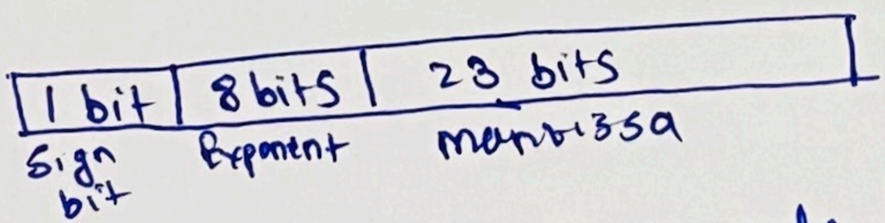
2's = add 1 in swapped number = 11010101
+00000001
11010110

This will get stored in memory → 11010110

10] Storing floats in memory :—

Case 1 :—

float f = 8.125f;



= 32-bit (float)

Step 1 :- find binary of left side of . (i.e. dot) = 8

8 = 1000

Step 2 :- find binary of right side of . (i.e. dot) = 0.125
to find binary of decimal we have to multiply it with 2

0.125 x 2 = 0.25 = 0
0.25 x 2 = 0.5 = 0
0.5 x 2 = 1.0 = 1

0.0 → as we kept 1 in above step our flow came to 0.0 that is the end of the flow.

So, binary of 0.125 = 001

So, binary of 8.125 = 1000.001 (44)

Step 3 :- make it in the form of —

$(1.xx) \times 2^{exp}$

$= 1.000001 \times 2^3$ → (we moved . 3 place to left i.e. why 2^3)

Step 4 :— add bias to expoment

Bias = our exponent is 8 bit i.e. of 1 byte

So even if our exponent storgs -127 we will standardly add 127 in it to make that number 0 or greater than 0. it is use to avoid negative numbers we generally add bias as 127

So,

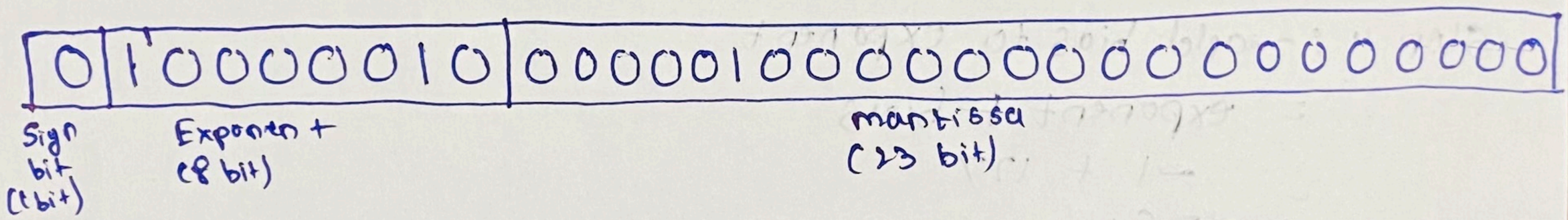
bias + exponent

= 127 + 3

= 130

= 10000010 (binary of 130)

Step 5 :- Place value in memory



Sign bit = Our number is posibive so, 0

Exponent = we found exponent in step 4, 10000010

mantissa = from step 3, .000001 will be considered as mantissa
This is how number is stored

Retrieving from memory :—

System.out.println(F);

$(-1)^{sign} * (1 + mantissa) * 2^{exp-bias}$

$(-1)^0 * (1 + mantissa) * 2^{130-127=3}$ exp = 10000010

$= 1 * (1 + mantissa) * 8$ mantissa = 000001
$2^{-1}\ 2^{-2}\ 2^{-3}\ 2^{-4}\ 2^{-5}\ 2^{-6}$
i.e. found where it is 1 appears in mantissa

$= 1 * (1 + 2^{-6}) * 8$

$= (1 + \frac{1}{64}) * 8$

$= (1 + 0.015625) * 8$

$= 1.015625 * 8 =$ **8.125**

Case 2 :—

float f = 0.7f;

Step 1 :- Binary of left side of . (i.e. dot) = 0

Step 2 :- Binary of right side of . (i.e. dot) = 0.7

$0.7 \times 2 = 1.4 = 1$

$0.4 \times 2 = 0.8 = 0$

$0.8 \times 2 = 1.6 = 1$

$0.6 \times 2 = 1.2 = 1$

$0.2 \times 2 = 0.4 = 0$

$0.4 \times 2 = 0.8 = 0$

$0.8 \times 2 = 1.6 = 1$

$0.6 \times 2 = 1.2 = 1$

$0.2 \times 2 = 0.4 = 0$

Repeating terms

So, binary of 0.7 = 10110011 0....

So, 0.7

0.10110011 0....

Step 3 :- $(1.xx) * 2^{expo}$

$= 0.10110011 0.... * 2^{expo}$

$= 1.0110011 0 * 2^{-1}$ (-1 because we moved . to right)

Step 4 :- add bias to exponent

= exponent + bias

= -1 + 127

= 126

= 1111110 (binary of 126)

Step 5 :- Place value in memory

0	01111110	0110 0110 0110 0110 0110 011
Sign bit (1 bit)	exponent (8 bit)	mantissa (23 bit)

Sign bit → positive number so, 0

exponent → from step 4, 01111110

mantissa → Repeating from step 3, so continued till 23 bits filled

So, this is how

float 0.7 stored in memory

Retreiving from memory :-

System.out.println(f);

$= (-1)^{sign} \times (1 + mantissa) \times 2^{exp-bias}$

$= (-1)^{0} \times (1 + mantissa) \times 2^{126-127 = -1}$

$= 1 \times (1 + mantissa) \times 2^{-1}$

$= (1 + mantissa) \times \frac{1}{2}$ —— : eqn (1)

Finding mantissa,

-2 -3 -6 -7 -10 -11 -14 -15 -18 -19 -22 -23

0 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1 - (2)

put eqn (2) in eqn (1)

$= \left(1 + 2^{-2} + 2^{-3} + 2^{-6} + 2^{-7} + 2^{-10} + 2^{-11} + 2^{-14} + 2^{-15} + 2^{-18} + 2^{-19} + 2^{-22} + 2^{-23}\right) \times \frac{1}{2}$

$= \left(1 + \frac{1}{2^2} + \frac{1}{2^3} + \frac{1}{2^6} + \frac{1}{2^7} + \frac{1}{2^{10}} + \frac{1}{2^{11}} + \frac{1}{2^{14}} + \frac{1}{2^{15}} + \frac{1}{2^{18}} + \frac{1}{2^{19}} + \frac{1}{2^{22}} + \frac{1}{2^{23}}\right) \times \frac{1}{2}$

$= \frac{1.39999997615814208984375}{2}$

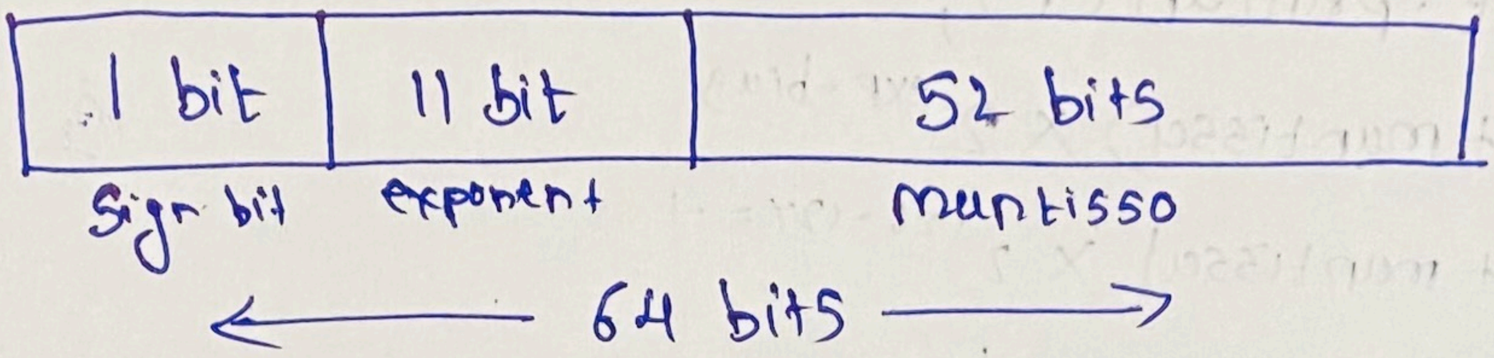
$f = 0.69999998807907104421875$ —— (3)

So, when we retrieve value of f like

System.out.printf("%.20f%n", f);

then, we will get answer like in eqn 3

11] Storing double in memory (64 bit) :



Note : bias = $(2^{11-1} - 1)$

$= 2^{10} - 1$

$= 1023$

Everything is same as float

Note :-

float & double sometimes causes data loss because of repeating terms in matissa. So java comes up with BigDecimal where data loss doesn't happen